

Final Exam**December 19 (Friday), 2025**

Department	Name	Score

Table 1: Student Information Table

(Python Basics)**Question 1** (2 points)

What will the following code print ?

```
def func(x, y, *args):  
    print(args, x)  
  
func(1, 2, 3, 'hello')
```

Question 2 (1 point)

What will this code print?

```
a= [x**2 for x in range(10)]  
print(a[2:4])
```

Question 3 (2 points)

In Python, decorators are functions that modify the behavior of other functions. Below is an example decorator. What will this code print?

```
def only_cat(func):  
    def wrapper(animal):  
        if animal != "cat":  
            print("no water")  
        else:  
            func(animal)  
    return wrapper  
  
@only_cat  
def drink_water(animal):  
    print(f"{animal} drinks water")  
  
drink_water("dog")  
drink_water("cat")
```

Question 4 (2 points)

Recall that lists in Python hold everything in memory, while generators in Python generate values on demand, making them more memory-efficient. What will this code print?

```
def f(x):  
    return x + 5  
  
L = [f(x) for x in range(3)]    # List comprehension  
G = (f(x) for x in range(3))    # Generator  
  
print(L)  
print(next(G))  
print(next(G))
```

Question 5 (5 points)

What is printed when the following code is executed? Write the output in the correct order, one line per output.

```
class SimpleModule:  
    def __init__(self, scale=2):  
        self.scale = scale  
  
    def __call__(self, x):  
        print("call")  
        x = self.forward(x)  
        print("done")  
        return x * self.scale  
  
class Func(SimpleModule):  
    def forward(self, x):  
        print("forward")  
        return x + 1  
  
m = Func(scale=3)  
print(f"Result: {m(5)}")
```

(Understanding PyTorch)

Question 6 (2 points)

What will this code print?

```
import torch

A = torch.tensor([[1, 2],
                  [3, 4],
                  [5, 6]])

B = torch.tensor([[[1, 2, 3],
                  [4, 5, 6]],
                  [[7, 8, 9],
                  [10, 11, 12]]])

print(A.shape)
print(B.shape)
```

Question 7 (3 points)

Consider the following PyTorch code. What will this code print?

```
import torch

a = torch.arange(3)

b = a.unsqueeze(0)
c = a.unsqueeze(1)

print(b.shape)
print(c.shape)
```

Question 8 (5 points)

What will this code print? (Hint: In the code below, `x = torch.randn(4, 3)` represents a batch of 4 samples, where the last dimension (3) corresponds to the number of input features per sample.)

```
import torch
import torch.nn as nn

layer = nn.Linear(in_features=3, out_features=2)
x = torch.randn(4, 3)

# Hint: Apply a linear transformation  $y = x W^T + b$  to the input  $x$ 
y = layer(x)

print(y.shape)
print(layer.weight.shape) # shape of  $W$  in  $y = x W^T + b$ 
print(layer.bias.shape)  # shape of  $b$  in  $y = x W^T + b$ 
```

(Understanding Convolutional Neural Network (CNN))**Question 9** (5 points)

In the following code, `print(y.shape)` produces `torch.Size([8, 16, 32, 32])`. Please explain how the last two values, 32 and 32 in `torch.Size([8, 16, 32, 32])` are obtained.

```
import torch
import torch.nn as nn

x = torch.randn(8, 3, 64, 64) # batch=8, RGB image 64x64

conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=4,
                  stride=2, padding=1)
y = conv(x)

print(y.shape) # torch.Size([8, 16, 32, 32]) why ?
```

(Understanding Natural Language Processing (NLP))

Question 10 (5 points)

Consider the following Regular Expression code. What will this code print?

```
import re
text = "Hello, world !"

a = re.findall(r"[A-Za-z]+", text)
# Hint: [A-Za-z]+ matches one or more alphabetic words

b = re.findall(r"[^\w\s]", text)
# Hint: [^\w\s] matches any character that is NOT a word character (\w)
# and NOT whitespace (\s)

print(a)
print(b)
```

Question 11 (8 points)

The CBOW (Continuous Bag of Words) model is a popular architecture used in Word2Vec.

- (a) (4 points) Given a center word w_c , what is the training objective of CBOW ?
- (b) (4 points) Explain, in your own words, what is the negative sampling in CBOW.

Question 12 (10 points)

Let $\mathbf{a}$ and $\mathbf{b}$ be *unit vectors* in $\mathbb{R}^n$ (i.e., $\|\mathbf{a}\| = 1$ and $\|\mathbf{b}\| = 1$). Suppose the angle between them is θ (with $0 \leq \theta \leq \pi$). Prove that

$$\mathbf{a} \cdot \mathbf{b} = \cos \theta.$$

Question 13 (15 points)

Consider scaled dot-product attention for a single query vector and two key-value pairs:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where the softmax is applied row-wise (here it is over the two scores). Assume $d_k = 2$.

Let the query be

$$Q = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

Let the two keys and two values be

$$K_1 = \begin{bmatrix} 2 & 1 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 0 & 1 \end{bmatrix},$$

$$V_1 = \begin{bmatrix} 1 & 2 \end{bmatrix}, \quad V_2 = \begin{bmatrix} 3 & 0 \end{bmatrix}.$$

1. (5 points) Compute the scaled scores

$$s_1 = \frac{QK_1^\top}{\sqrt{2}}, \quad s_2 = \frac{QK_2^\top}{\sqrt{2}}.$$

2. (5 points) Compute the attention weights

$$\alpha_1 = \frac{e^{s_1}}{e^{s_1} + e^{s_2}}, \quad \alpha_2 = \frac{e^{s_2}}{e^{s_1} + e^{s_2}}.$$

3. (5 points) Compute the attention output

$$Y = \alpha_1 V_1 + \alpha_2 V_2.$$

Question 14 (10 points)

The Transformer architecture relies heavily on the attention mechanism. However, attention alone does not encode information about the order of tokens. Explain how positional encoding is incorporated into the Transformer input.

(Understanding Diffusion Models)

Question 15 (15 points)

(a) (10 points) Explain the core idea of a diffusion model. What are the roles of the **forward process** and the **reverse process**?

(b) (5 points) State **advantages** of diffusion models compared to GANs and VAEs.

Question 16 (10 points)

Why do we use `transpose(-2, -1)` in attention (see the code, `scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d)`)?

```

import math
import torch
import torch.nn.functional as F

def simple_attention(q, k, v):
    """
    Scaled dot-product attention (single-head, no mask).

    Args:
    q, k, v: tensors of shape (B, L, D)
    B = batch size
    L = sequence length
    D = embedding dimension

    Returns:
    out: (B, L, D) attention output
    attn: (B, L, L) attention weights (row-wise softmax)
    """
    d = q.size(-1)

    # 1) Scores: (B, L, D) @ (B, D, L) -> (B, L, L)
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d)

    # 2) Weights: row-wise softmax over keys (last dimension)
    attn = F.softmax(scores, dim=-1)

    # 3) Output: (B, L, L) @ (B, L, D) -> (B, L, D)
    out = torch.matmul(attn, v)
    return out, attn

# Small runnable example -----
torch.manual_seed(0)

B, L, D = 1, 3, 4
q = torch.randn(B, L, D)
k = torch.randn(B, L, D)
v = torch.randn(B, L, D)

out, attn = simple_attention(q, k, v)

```